Linear data structures

22A. Linear Data Structures

Questions: 7 **Difficulty:** Junior to Mid

Q 22.1 Arrays — what are the time complexities and frontend use cases?

Difficulty: Junior **Asked at:** Every company

Arrays are the most used data structure in frontend. Every list, grid, table, and collection is an array.

Time complexities:

JAVASCRIPT Copy

```
Operation          | Time       | Notes
-------------------|------------|-----------------------------------
Access by index    | O(1)       | arr[5] - instant
Push (end)         | O(1)*      | Amortized - occasional resize
Pop (end)          | O(1)       |
Unshift (start)    | O(n)       | Shifts every element right
Shift (start)      | O(n)       | Shifts every element left
Insert (middle)    | O(n)       | splice - shifts elements
Delete (middle)    | O(n)       | splice - shifts elements
Search (unsorted)  | O(n)       | Linear scan
Search (sorted)    | O(log n)   | Binary search
Sort               | O(n log n)| Tim Sort in V8
```

Frontend-relevant patterns:

JAVASCRIPT Copy

```javascript
// Immutable update (React state) — spread creates a new array
const newItems = [...items, newItem];                          // Add to end
const filtered = items.filter(item => item.id !== removeId); // Remove by
const updated = items.map(item =>
  item.id === targetId ? { ...item, done: true } : item       // Update one
);

// When arrays are the wrong choice:
// Frequent lookups by key → use Map or object
// Checking membership → use Set
// Need O(1) delete from middle → use linked list or Map
```

Follow-up: "When would you use a Map instead of an array?"

When you need O(1) lookup by key. An array of users where you frequently find by `userId` is O(n) per lookup. A `Map<string, User>` is O(1). Convert the array to a Map for repeated lookups, or use both (array for ordered rendering, Map for lookups).

Q 22.2 Strings — what manipulation patterns matter for UI work?

Difficulty: Junior **Asked at:** Amazon, Flipkart, Atlassian

Strings in JavaScript are immutable — every manipulation creates a new string. This matters for performance when building large strings incrementally.

Common frontend string operations:

JAVASCRIPT Copy

```
  if (str.length ≤ maxLength) return str;
  return str.slice(0, maxLength - 3) + '...';
}

// Slug generation (URLs from titles)
function slugify(text) {
  return text
    .toLowerCase()
    .replace(/[^\w\s-]/g, '')     // Remove special chars
    .replace(/\s+/g, '-')         // Spaces to hyphens
    .replace(/-+/g, '-')          // Collapse multiple hyphens
    .trim();
}
// "Hello World! 2026" → "hello-world-2026"

// Search highlighting (marking matched text)
function highlightMatch(text, query) {
  if (!query) return text;
  const regex = new RegExp(`(${escapeRegex(query)})`, 'gi');
  return text.replace(regex, '<mark>$1</mark>');
}

// Template literal for HTML generation (careful - XSS risk)
// Use textContent for user input, innerHTML only with sanitization
```

Performance note: String concatenation in a loop (`str += chunk`) is optimized in modern V8 — but for very large strings (100KB+), using an array and `.join('')` at the end is measurably faster.

Q 22.3 Stacks — where do they appear in frontend?

Difficulty: Junior **Asked at:** Amazon, Flipkart, Google

A stack is LIFO (Last In, First Out). JavaScript arrays work as stacks with `push()` and `pop()`.

JAVASCRIPT Copy

```
  #items = [];
  push(item) { this.#items.push(item); }
  pop() { return this.#items.pop(); }
  peek() { return this.#items[this.#items.length - 1]; }
  isEmpty() { return this.#items.length === 0; }
  get size() { return this.#items.length; }
}
```

Frontend use cases:

- **Undo/Redo** — undo stack holds past states, redo stack holds undone states (Chapter 9, Q9.11)
- **Browser history** — back button pops from history stack
- **Balanced parentheses** — validating code/math expressions
- **Function call stack** — how JavaScript executes nested function calls
- **DFS traversal** — iterative depth-first search uses an explicit stack

---

Q 22.4 Queues — where do they appear in frontend?

Difficulty: Junior **Asked at:** Amazon, Flipkart

A queue is FIFO (First In, First Out). JavaScript doesn't have a built-in Queue class, but arrays work with `push()` + `shift()` (though `shift()` is O(n)).

JAVASCRIPT Copy

```
// Efficient queue using object (O(1) for both enqueue and dequeue)
class Queue {
  #items = {};
  #head = 0;
  #tail = 0;

  enqueue(item) { this.#items[this.#tail++] = item; }
  dequeue() {
    if (this.isEmpty()) return undefined;
```

```
    }
    peek() { return this.#items[this.#head]; }
    isEmpty() { return this.#tail === this.#head; }
    get size() { return this.#tail - this.#head; }
}
```

Frontend use cases:

- **Task queues** — process API calls in order, rate-limited notification delivery
- **BFS traversal** — breadth-first DOM traversal, level-order tree traversal
- **Message queues** — WebSocket message processing in order
- **Animation queues** — sequence animations one after another
- **Print queue** — process print requests FIFO

Q 22.5 Linked Lists — when are they better than arrays?

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart

Linked lists store elements as nodes where each node points to the next (and previous, in a doubly linked list). Arrays store elements contiguously in memory.

JAVASCRIPT Copy

```
class ListNode {
  constructor(value) {
    this.value = value;
    this.next = null;
  }
}

class LinkedList {
  constructor() {
    this.head = null;
    this.tail = null;
```

```
  addToEnd(value) {
    const node = new ListNode(value);
    if (!this.head) {
      this.head = this.tail = node;
    } else {
      this.tail.next = node;
      this.tail = node;
    }
    this.size++;
  }

  removeFirst() {
    if (!this.head) return null;
    const value = this.head.value;
    this.head = this.head.next;
    if (!this.head) this.tail = null;
    this.size--;
    return value;
  }
}
```

When linked lists beat arrays: O(1) insertion/deletion at known positions (given a node reference). An LRU Cache combines a doubly linked list (for O(1) move-to-front) with a hash map (for O(1) lookup). This is the most common interview use case.

Frontend relevance: LRU caches for API responses, undo/redo history with limited size, playlist/carousel navigation (circular linked list), and text editor cursor movement (doubly linked list).

Q 22.6 Doubly Linked Lists — why do LRU caches use them?

Difficulty: Mid **Asked at:** Google, Amazon, Flipkart

A doubly linked list has both `next` and `prev` pointers. This allows O(1) removal of any node (given a reference) because you can update both neighbors without traversing

from the head.

JAVASCRIPT

Copy

```
class DoublyLinkedList {
  constructor() {
    // Sentinel nodes simplify edge cases (no null checks)
    this.head = { prev: null, next: null };
    this.tail = { prev: null, next: null };
    this.head.next = this.tail;
    this.tail.prev = this.head;
  }

  addToFront(node) {
    node.next = this.head.next;
    node.prev = this.head;
    this.head.next.prev = node;
    this.head.next = node;
  }

  remove(node) {
    node.prev.next = node.next;
    node.next.prev = node.prev;
  }

  removeLast() {
    if (this.tail.prev === this.head) return null;
    const node = this.tail.prev;
    this.remove(node);
    return node;
  }
}
```

LRU Cache = Doubly Linked List + Hash Map:

The list maintains access order (most recently used at front). The map provides O(1) key lookup to find the node. On access: find node via map → remove from current position → add to front. On eviction: remove from the tail (least recently used).

This is one of the most asked coding problems in interviews — see Chapter 25 (JS Problems SDE2).

Q 22.7 Circular Buffers — what are they used for?

Difficulty: Mid **Asked at:** Google, Amazon

A circular buffer (ring buffer) is a fixed-size array where the end wraps around to the beginning. When full, new data overwrites the oldest data.

JAVASCRIPT Copy

```
class CircularBuffer {
  constructor(capacity) {
    this.buffer = new Array(capacity);
    this.capacity = capacity;
    this.head = 0;
    this.size = 0;
  }

  push(item) {
    const index = (this.head + this.size) % this.capacity;
    this.buffer[index] = item;
    if (this.size === this.capacity) {
      this.head = (this.head + 1) % this.capacity;  // Overwrite oldest
    } else {
      this.size++;
    }
  }

  getAll() {
    const result = [];
    for (let i = 0; i < this.size; i++) {
      result.push(this.buffer[(this.head + i) % this.capacity]);
    }
    return result;
  }
}

// Keep last 100 log entries — older ones automatically discarded
const logBuffer = new CircularBuffer(100);
logBuffer.push({ timestamp: Date.now(), message: 'User clicked button' });
```

input buffers.

QUICK CHECK — no marks, just for you.

1. A user list component keeps 5,000 users in an array and, on every render, looks up a user by id with users.find(u => u.id === targetId) — hundreds of times per render. According to the lesson, what is the reason to build a Map<string, User> from that array instead?

- ○ A Map makes the data immutable, which lets React skip re-rendering the list
- ○ A Map stores its entries contiguously in memory, so iterating it is faster than iterating an array
- ○ A Map keeps its entries sorted by key, so lookups can use binary search at O(log n)
- ○ Each lookup drops from O(n) to O(1), so repeated find-by-key work stops scaling with the list length

Check answer

2. What does this snippet print?

JAVASCRIPT Copy

```
class CircularBuffer {
  constructor(capacity) {
    this.buffer = new Array(capacity);
    this.capacity = capacity;
    this.head = 0;
    this.size = 0;
  }

  push(item) {
    const index = (this.head + this.size) % this.capacity;
    this.buffer[index] = item;
    if (this.size === this.capacity) {
      this.head = (this.head + 1) % this.capacity;
```

```
  }

  getAll() {
    const result = [];
    for (let i = 0; i < this.size; i++) {
      result.push(this.buffer[(this.head + i) % this.capacity]);
    }
    return result;
  }
}

const buf = new CircularBuffer(3);
[1, 2, 3, 4, 5].forEach(n => buf.push(n));
console.log(buf.getAll().join(','));
console.log(buf.size, buf.head);
```

- ○ `1,2,3`
`3 0`
- ○ `4,5`
`2 0`
- ○ `3,4,5`
`3 2`
- ○ `1,2,3,4,5`
`5 0`

Check answer

3. An LRU cache is built from a doubly linked list plus a hash map. Per the lesson, what does the second (prev) pointer buy you that a singly linked list could not provide?

- ○ It stores the nodes contiguously in memory, so the cache gets O(1) access by index like an array
- ○ It keeps the nodes ordered by key, so evicting the least recently used entry becomes a binary search

- [ ] Removing an arbitrary node you already hold a reference to becomes O(1), since both neighbours can be relinked without traversing from the head

Check answer

Previous
Testing patterns

Next
Hash based structures

LEARN MORE

ARTICLES & DOCS

- Linked list (VisuAlgo)
  visualgo.net
- Data structures (GeeksforGeeks)
  geeksforgeeks.org

VIDEOS



- Arrays, linked lists, stacks & queues
  YouTube



- Introduction to linear data structures
  YouTube



- Stacks, queues & linked lists
  YouTube

Hand-picked references for this lesson.